大规模有限元并行组装算法研究报告：图着色、区域分解与新兴架构的演进

1. 绪论：高性能计算范式下的有限元组装瓶颈

1.1 有限元方法的计算演进与性能挑战

有限元方法（Finite Element Method, FEM）作为一种将连续介质物理问题离散化的数值分析技术，已成为现代工程与科学计算的基石。从航空航天结构的强度分析到地下水渗流的模拟，再到电磁场与多物理场耦合问题，FEM的应用范围无所不包。在过去几十年的发展历程中，随着摩尔定律推动的计算硬件性能指数级增长，有限元模拟的规模已从早期的数千个自由度（Degrees of Freedom, DoF）迅速扩展至如今的数亿甚至数百亿自由度。

在传统的串行计算或早期的并行计算模式（如单核CPU或早期的多核集群）中，计算流程的性能瓶颈通常位于线性方程组的求解（Linear Solver）阶段。这是因为求解大规模稀疏线性方程组 $\mathbf{Ku} = \mathbf{f}$ 涉及复杂的矩阵分解或迭代运算，其时间复杂度通常在 $\mathcal{O}(N^{1.2})$ 到 $\mathcal{O}(N^2)$ 之间。因此，学术界和工业界在过去的三十年中投入了巨大的资源研发高效并行求解器，例如基于多重网格（Multigrid）的预条件子、代数多重网格（AMG）以及基于区域分解（Domain Decomposition）的并行直接求解器（如MUMPS, SuperLU, PaStiX）。

然而，随着求解器算法的日益成熟以及高性能计算（HPC）硬件向异构众核（Heterogeneous Many-core）架构的演进，一个新的性能瓶颈逐渐浮现——整体刚度矩阵的组装（Global Stiffness Matrix Assembly）。在非线性分析（如弹塑性大变形）、显式动力学模拟或涉及频繁网格重构的自适应分析中，刚度矩阵的数值需要在每一个时间步或牛顿迭代步中重新计算与组装。研究表明，在某些极端工况下，特别是采用高阶单元或在GPU等高吞吐量架构上运行时，矩阵组装阶段的耗时占比已超过50%，甚至成为制约整体仿真效率的首要因素。这一现象标志着有限元计算已从“求解主导（Solver-Dominated）”向“组装主导（Assembly-Dominated）”的范式转变。

1.2 问题的本质：计算与访存的失衡

矩阵组装过程在现代硬件上面临着双重挑战：

第一，计算密集度与访存密集度的矛盾。单元刚度矩阵的计算通常涉及大量的浮点运算（数值积分、形函数求导），属于计算密集型任务；而将局部矩阵累加到全局矩阵的过程则是典型的访存密集型任务，且涉及非结构化的随机内存写入。随着处理器峰值浮点性能（FLOPS）的增长速度远超内存带宽（Memory Bandwidth）的增长速度（即著名的“内存墙”问题），组装过程中的数据搬运日益成为瓶颈。

第二，**并行环境下的写冲突（Write Conflict）**。在有限元网格中，相邻的单元共享公共节点。当数以千计的并行线程同时处理这些相邻单元并试图将各自的贡献累加到全局矩阵的同一位置时，就会发生数据竞争（Data Race）。如何高效地解决这一冲突，同时保持硬件的高并发度，是并行组装算法设计的核心难题。

本报告旨在详尽梳理当前针对稀疏存储格式的主流并行组装算法，重点对比分析图**着色算法（Graph Coloring）*与*基于区域分解思想的数据划分策略**，并探讨原子操作（Atomic Operations）与无矩阵（Matrix-Free）技术在现代硬件上的应用前景。报告将结合NVIDIA GPU与国产申威（Sunway）SW26010等前沿架构，揭示算法演进背后的硬件驱动力。

---

2. 并行组装的数学描述与稀疏存储架构

在深入探讨具体算法之前，必须建立严谨的数学描述，并理解稀疏矩阵存储格式对并行操作的底层约束。

2.1 刚度矩阵组装的代数映射

有限元分析将物理域 Ω 离散化为互不重叠的单元集合 $\{e_1, e_2, \ldots, e_{N_e}\}$。对于任意一个单元 e，其局部刚度矩阵 $\mathbf{K}^e$ 是一个 $N_{local} \times N_{local}$ 的稠密矩阵，其中 $N_{local} = n_{en} \times d$（$n_{en}$ 为单元节点数，d 为节点自由度数）。

整体刚度矩阵 $\mathbf{K}^G$ 的构建在数学上可以表示为所有单元刚度矩阵经过布尔映射后的累加：

$$\mathbf{K}^G = \sum_{e=1}^{N_e} \mathbf{L}_e^T \mathbf{K}^e \mathbf{L}_e$$

其中，$\mathbf{L}_e$ 是一个 $N_{local} \times N_{global}$ 的布尔映射矩阵（Boolean Mapping Matrix），用于将单元的局部自由度索引映射到全局自由度索引。

在计算机实现的微观层面，这一过程转化为对全局内存地址的并发更新。对于单元 e 的局部元素 K_{ij}^e，其对应的全局行索引为 $I = \text{map}(e, i)$，全局列索引为 $J = \text{map}(e, j)$。组装操作即执行：

$$\mathbf{K}_{IJ}^G \leftarrow \mathbf{K}_{IJ}^G + K_{ij}^e$$

由于网格拓扑的连通性，对于任意内部节点，通常有多个单元共享该节点。例如，在三维非结构化四面体网格中，一个节点可能被10到20个单元共享。这意味着在并行计算中，可能有10到20个线程试图同时更新 $\mathbf{K}_{IJ}^G$。若缺乏同步机制，这种并发写入将导致结果的不确定性甚至错误。

2.2 稀疏存储格式对并行性的决定性影响

整体刚度矩阵 $\mathbf{K}^G$ 的稀疏性（Sparsity）要求采用特定的压缩存储格式。不同的格式对并行插入操作的兼容性差异巨大。

2.2.1 坐标格式（COO）与并行的天然亲和性

坐标格式（Coordinate List, COO）存储三元组序列 $(row_idx, col_idx, value)$。

- **并行优势**：COO格式允许“追加”式写入。每个线程可以独立计算单元刚度矩阵，并将生成的三元组直接写入预分配的显存数组中。由于不涉及对同一内存地址的累加（此时重复的 (I, J) 条目共存），不存在写冲突。
- **存储劣势**：COO格式存储了大量重复的索引，且未合并的数值导致内存占用显著增加（通常是最终CSR格式的3-5倍）。此外，后续的线性求解器通常不支持COO格式，因此必须进行昂贵的排序（Sort）和压缩（Compress）操作。

2.2.2 压缩稀疏行格式（CSR）的并行困境

CSR格式通过 `values`、`col_indices` 和 `row_ptr` 数组存储。它是求解器最常用的格式。

- **组装难题**：直接并行组装CSR极具挑战性。首先，必须预先确知每一行的非零元数量以构建 `row_ptr`（需要符号分析）。其次，即使结构已知，将数值累加到CSR中需要在线性存储的 `col_indices` 数组中搜索列索引。当多个线程同时搜索并更新同一行时，不仅存在原子竞争，还会造成极高的缓存未命中率（Cache Misses），因为CSR的紧凑存储破坏了基于单元几何邻近性的内存局部性。

2.2.3 ELLPACK与混合格式（HYB）

为了适应GPU的SIMD（单指令多数据）架构，ELL格式将稀疏矩阵填充为固定宽度的稠密矩阵，以保证合并访问。

- **适用性**：对于结构化网格，ELL效率极高。但对于非结构化网格，节点连接度的差异会导致大量的零填充，浪费宝贵的显存带宽。

---

3. 图着色算法（Graph Coloring）：同步与隔离的艺术

在GPU计算的早期（如NVIDIA Tesla架构）以及多核CPU并行化的初期，硬件缺乏高效的原子操作支持。为了解决写冲突，基于图论的着色算法成为了主流解决方案。其核心逻辑是通过拓扑分析，将存在数据依赖的任务在时间域上强制隔离。

3.1 算法的数学原理与图论抽象

图着色算法将有限元网格的连通性抽象为一个对偶图（Dual Graph）$G = (V, E)$。

- **顶点** V：图中的每个顶点代表一个有限元单元。
- **边** E：若两个单元 e_i 和 e_j 共享至少一个节点（即它们会竞争写入同一个全局矩阵位置），则在它们之间建立一条边。

算法的目标是将顶点集合 V 划分为 k 个互不相交的颜色集（Color Sets）$\{S_1, S_2, \ldots, S_k\}$，满足：

$$\forall e_a, e_b \in S_c, \quad (e_a, e_b) \notin E$$

这意味着同一颜色集内的任意两个单元互不相邻，不存在共享节点。因此，在处理某个颜色集 S_c 时，所有线程可以安全地并行执行，无需任何锁或原子操作。

3.2 典型着色算法与实施流程

最常用的着色算法是贪心算法（Greedy Strategy），具体包括：

1. **First-Fit（首次适应算法）**：遍历所有单元，将其分配给第一个不与邻居冲突的颜色。该算法简单快速，但生成的颜色数（Chromatic Number）可能并非最优。
2. **DSatur（度饱和算法）**：优先为那些约束最紧（即已着色邻居最多）的单元着色，通常能产生更少的颜色数，但计算复杂度较高。
3. **并行推测着色（Parallel Speculative Coloring）**：如Gebremedhin-Manne算法，允许并行线程尝试着色，随后检测冲突并重做，适合在GPU上并行生成色图。

并行组装流程：

1. **预处理**：构建对偶图并进行着色。
2. **分批执行**：外层循环遍历颜色 $c = 1 \dots k$。
3. **内核启动**：启动GPU内核，仅计算属于颜色 c 的单元。
4. **全局屏障**：在颜色切换间隙插入全局内存屏障（Global Memory Barrier），确保当前颜色的所有写入完成且可见。

3.3 性能局限性与现代硬件的不适应性

尽管图着色法在逻辑上完美解决了冲突，且保证了数值结果的确定性（Determinism），但在现代高性能硬件上，其性能短板日益凸显。

3.3.1 并行度震荡与“长尾效应”

为了避免冲突，网格被分割成多个颜色组。在任意时刻，只有一部分单元被处理。

- **低占有率（Low Occupancy）**：对于三维非结构化网格，特别是四面体网格，节

点的连接度较高，往往需要20种甚至更多的颜色。这意味着平均并行度降低为原来的 1/20。

- **负载不均衡**：不同颜色集的单元数量往往差异巨大。早期的颜色集单元众多，能占满GPU；而后期的颜色集可能仅包含少量“残余”单元，导致GPU的大量流多处理器（SM）闲置，出现明显的性能“长尾”。

3.3.2 空间局部性的彻底破坏

这是图着色法最致命的弱点。

- **物理离散导致访存离散**：同一颜色集中的单元在几何空间上通常是互不相邻且随机分布的。当GPU的一个线程束（Warp, 32个线程）处理一种颜色的单元时，这32个线程需要访问的节点数据和矩阵行索引在内存中相距甚远。
- **缓存抖动（Cache Thrashing）**：这种极其离散的内存访问模式导致无法利用GPU的合并访问（Coalesced Access）机制，L1/L2缓存命中率急剧下降。实验表明，相比于按几何顺序处理，着色法的数据加载带宽效率可能下降60%-80%。

---

4. 区域分解算法与硬件级数据划分：申威架构的启示

虽然“区域分解”通常指代代数求解器中的Schwarz交替法，但在并行组装语境下，它体现为**基于空间拓扑的任务与数据划分**。特别是在国产申威（Sunway）SW26010众核处理器上，由于其独特的硬件架构，区域分解思想被下沉到了芯片微观层面，成为解决组装瓶颈的关键。

4.1 申威SW26010架构特征与挑战

“神威·太湖之光”超算的核心处理器SW26010采用异构众核架构，包含4个核组（Core Groups）。每个核组由1个管理处理核心（MPE）和64个计算处理核心（CPE）组成。

- **无缓存一致性**：CPE阵列没有传统的L1/L2缓存一致性硬件。取而代之的是用户可显式控制的64KB高速暂存器（LDM, Local Data Memory）。
- **显式通信**：核心间的数据交换不依赖共享内存的窥探机制，而是通过直接内存访问（DMA）和寄存器通信网络。

这种架构使得传统的图着色或原子操作策略难以直接应用：图着色需要频繁的全局同步（开销极大），而原子操作在无缓存一致性的主存上效率极低。因此，必须采用**微观区域分解（Micro-Domain Decomposition）**策略。

4.2 基于微观区域分解的并行组装策略（swParaFEM）

Xue等人（2018, 2020）提出的swParaFEM框架，将区域分解思想极致地应用于组装过程。

4.2.1 显式数据划分与映射

算法首先将分配给MPE的网格块（Macro-Domain），进一步显式地划分为64个子块（Micro-Subdomains），每个子块映射到一个CPE上。这实际上是在芯片内部进行了一次二级的区域分解。

- **DMA预取**：利用DMA引擎，每个CPE异步地将自己负责的子块单元数据从主存拉取到私有的LDM中。由于数据在LDM中是紧凑存储的，计算核心可以极高带宽地读取数据，掩盖了主存延迟。

4.2.2 界面处理：寄存器通信替代原子锁

在区域分解中，核心难点在于子块边界上的节点（Interface Nodes）。这些节点由多个CPE共享，必须累加各方的贡献。

- **传统做法**：各CPE将结果写回主存，利用原子锁解决冲突。这在SW26010上不仅慢，而且容易造成总线拥塞。
- **寄存器通信优化**：swParaFEM利用CPE阵列特有的行/列寄存器通信网络。
    a. 每个CPE先在LDM中完成内部节点的累加（无冲突）。
    b. 对于边界节点，CPE之间通过寄存器直接传递数据包。例如，同一行的CPE可以通过行总线将边界节点的贡献值传递给“归约主CPE”（Owner CPE）。
    c. 归约主CPE完成累加后，再统一写回主存。

这种策略将“内存竞争”转化为了“确定性的通信调度”，极大地降低了访存延迟。实验显示，该方法在SW26010上的组装效率达到了峰值性能的20%-30%，且具有极好的强扩展性，是区域分解思想与特定硬件架构结合的典范。

---

5. 原子操作的硬件革命：从串行化到Warp聚合

随着GPU架构从Fermi向Kepler、Pascal乃至Volta、Ampere演进，硬件层面对原子操作（Atomic Operations）的支持发生了质的飞跃。现代原子操作不再是性能杀手，而是成为了并行组装的首选方案，逐渐取代了图着色法。

5.1 硬件演进：从锁机制到L2原子算术单元

在早期的GPU架构中，`atomicAdd` 指令会被编译为复杂的“读取-锁定-修改-写入-解锁”序列，导致执行流水线的严重停顿（Serialization）。如果多个线程竞争同一地址，性能会呈指数级下降。

从 **Pascal** 架构开始，NVIDIA引入了位于L2缓存子系统中的专用原子算术单元（Atomic ALUs）。

- **近存计算**：原子操作不再需要将数据拉回流多处理器（SM）进行加法，而是直接在L2缓存处完成。这大幅降低了延迟和总线流量。
- **FP64支持**：Pascal及后续架构原生支持64位双精度浮点数（double）的原子加法，彻底扫清了科学计算的障碍。
- **发后即忘**：现代原子操作在低冲突率下表现为“发后即忘（Fire-and-Forget）”指令，吞吐量已接近常规存储指令。

5.2 Warp聚合（Warp Aggregation）：消除局部冲突

尽管硬件已大幅优化，但如果大量线程（如处理汇聚于一点的多个单元）同时冲击同一全局地址，仍会产生热点。为了进一步优化，研究者提出了**Warp聚合**技术。

该技术利用GPU线程束内（Intra-Warp）的洗牌指令（Shuffle Instructions, __shfl_down_sync）在寄存器层级解决局部冲突。

算法逻辑：

1. **冲突检测**：线程束内的线程首先比对各自的目标写入地址（I, J）。
2. **寄存器归约**：如果发现多个线程指向同一地址（例如线程$0, 1, 5$都要写入位置`idx`），它们利用Shuffle指令在寄存器间交换数值并求和。
3. **领头写入**：由该组中的一个“领头线程”（Leader Thread）代表全组执行一次全局`atomicAdd`。

优势：

这种方法将冲突消解在寄存器层级（零延迟），显著减少了对显存系统的原子请求次数。NASA的FUN3D团队在A100 GPU上的测试表明，基于Warp聚合的策略比传统的全局原子法快2倍以上，且完全无需图着色所需的预处理。

---

6. 排序归约与无矩阵方法：算法范式的转移

除了上述基于“冲突消解”的策略外，还有两类算法试图从根本上改变组装的流程。

6.1 排序与归约（Sort-and-Merge）：通用但昂贵

对于拓扑结构剧烈变化或无法预先符号分析的场景（如断裂力学中的网格重构），基于排序的算法提供了最强的鲁棒性。

1. **扩展（Expansion）**：各线程独立计算并将所有三元组 (I, J, val) 写入一个巨大的线性缓冲区（COO格式）。
2. **排序（Sort）**：利用GPU高效的基数排序（Radix Sort），以 (I, J) 为键对缓冲区进行排序。排序后，针对同一位置的所有贡献在物理内存中相邻。
3. **归约（Reduction）**：通过并行扫描（Scan）合并相邻的重复键值，生成唯一的非零元。
4. 压缩：直接输出为CSR格式。

- 评价：该方法实现了完全的负载均衡和合并访问，不存在线程发散。但其显存消耗是巨大的（通常需要存储比最终矩阵大10-20倍的中间数据），这往往限制了其在大规模问题上的应用。

6.2 无矩阵方法（Matrix-Free）：突破内存墙的终极方案

随着“内存墙”问题加剧，稀疏矩阵向量乘（SpMV）的低算术强度（Arithmetic Intensity, 约 0.2 FLOPs/Byte）使得现代GPU的算力被严重浪费。**无矩阵方法**主张永不显式组装全局矩阵。

当求解器需要计算 $\mathbf{v} = \mathbf{K}^G \mathbf{u}$ 时，直接利用单元积分定义在局部计算：

$$\mathbf{v} = \sum_e \mathbf{L}_e^T \left(\mathbf{K}^e (\mathbf{L}_e \mathbf{u}) \right)$$

核心优势：张量收缩（Tensor Contraction）

对于高阶单元（High-Order Elements, $p \geq 4$），形函数具有张量积结构。利用这一性质，可以将三维积分分解为一维张量收缩运算（Sum Factorization）。这种方法的算术强度可提升至 10-50 FLOPs/Byte，能够充分利用GPU的峰值浮点性能。

deal.II 库的最新研究表明，无矩阵方法在十亿级自由度问题上比传统CSR方法快 5-10 倍，且内存消耗减少 90%。

---

7. 比较分析与结论

表1总结了不同组装策略在关键指标上的对比。

结论：

有限元并行组装算法的演进是一部与硬件架构斗争的历史。

1. **图着色算法**在多核时代早期是王者，但在现代深存储层次结构的GPU上，其破坏缓存局部性的副作用已超过了其消除冲突的收益。
2. **区域分解算法**在微观层面的应用（如申威swParaFEM），证明了显式控制数据流动比依赖硬件缓存一致性更具扩展性。
3. 对于主流商用GPU，**Warp聚合的原子操作**已成为处理低阶单元的事实标准；而对于高阶单元，**无矩阵方法**代表了未来的方向，它通过数学层面的重构（张量收缩）彻底规避了组装带来的内存瓶颈。

未来的研究将趋向于混合精度组装（利用FP16/BF16加速计算）以及利用机器学习辅助预测非零元模式，以进一步逼近硬件的极限性能。